

# Mu Runtime Reference

version 0.2.21

## type keywords and aliases

|                      |                                            |                                   |
|----------------------|--------------------------------------------|-----------------------------------|
| <i>supertype</i>     | <i>T</i>                                   |                                   |
| <i>bool</i>          | <i>()</i> , <i>nil</i>                     | are false, else true              |
| <i>condition</i>     | <i>keyword</i>                             | see <b>exceptions</b>             |
| <i>list</i>          | <i>:cons</i> or <i>()</i> , <i>nil</i>     |                                   |
| <i>ns</i>            | <i>#s(:ns #(:t fixnum symbol))</i>         |                                   |
| <i>ns-designator</i> | <i>ns</i>                                  |                                   |
| <i>:null</i>         | <i>()</i> , <i>nil</i>                     |                                   |
| <i>:char</i>         | <i>char</i>                                | 8 bit ASCII                       |
| <i>:cons</i>         | <i>cons</i> , <i>list</i>                  | list, cons, dotted pair           |
| <i>:fixnum</i>       | <i>fixnum</i> , <i>fix</i>                 | 56 bit signed integer             |
| <i>:float</i>        | <i>float</i> , <i>fl</i>                   | 32 bit IEEE float                 |
| <i>:func</i>         | <i>function</i> , <i>fn</i>                | function                          |
| <i>:keyword</i>      | <i>keyword</i> , <i>key</i>                | symbol                            |
| <i>:stream</i>       | <i>stream</i>                              | file or string type               |
| <i>:struct</i>       | <i>struct</i>                              | see <b>structs</b>                |
| <i>:symbol</i>       | <i>symbol</i> , <i>sym</i>                 | LISP-1 symbol                     |
| <i>:vector</i>       | <i>vector</i> , <i>string</i> , <i>str</i> | typed vector                      |
|                      | <i>:bit</i> <i>:char</i> <i>:t</i>         |                                   |
|                      | <i>:</i>                                   | byte <i>:fixnum</i> <i>:float</i> |

## core

|                             |               |                                   |
|-----------------------------|---------------|-----------------------------------|
| <b>apply</b> <i>fn list</i> | <i>T</i>      | apply <i>fn</i> to <i>list</i>    |
| <b>compile</b> <i>form</i>  | <i>T</i>      | <i>mu</i> form compiler           |
| <b>eq</b> <i>T T'</i>       | <i>bool</i>   | <i>T</i> and <i>T'</i> identical? |
| <b>eval</b> <i>form</i>     | <i>T</i>      | evaluate <i>form</i>              |
| <b>type-of</b> <i>T</i>     | <i>key</i>    | type keyword                      |
| <b>view</b> <i>T</i>        | <i>vector</i> | vector of object                  |
| <b>fix</b> <i>fn T</i>      | <i>T</i>      | fixpoint of <i>fn</i>             |
| <b>gc</b>                   | <i>bool</i>   | garbage collection                |
| <b>repr</b> <i>T</i>        | <i>vector</i> | tag representation                |
| <b>unrepr</b> <i>vector</i> | <i>T</i>      | tag representation                |

tag *vector* is an 8 element *:byte* vector of little-endian argument tag bits.

## special forms

|                              |                 |                     |
|------------------------------|-----------------|---------------------|
| <i>:lambda list . list'</i>  | <i>function</i> | anonymous <i>fn</i> |
| <i>:alambda list . list'</i> | <i>function</i> | anonymous <i>fn</i> |
| <i>:quote T</i>              | <i>list</i>     | quoted form         |
| <i>:if T T' T''</i>          | <i>T</i>        | conditional         |

## frames

frame binding: (*fn* . #(:t ...))

|                                 |              |                                          |
|---------------------------------|--------------|------------------------------------------|
| <b>%frame-stack</b>             | <i>list</i>  | active frames                            |
| <b>%frame-pop</b> <i>fn</i>     | <i>frame</i> | pop <i>function</i> 's top frame binding |
| <b>%frame-push</b> <i>frame</i> | <i>cons</i>  | push frame                               |
| <b>%frame-ref</b> <i>fn fix</i> | <i>T</i>     | <i>function</i> , offset                 |

## symbols

|                                    |                             |                          |
|------------------------------------|-----------------------------|--------------------------|
| <b>boundp</b> <i>sym</i>           | <i>bool</i>                 | is <i>symbol</i> bound?  |
| <b>make-symbol</b> <i>string</i>   | <i>sym</i>                  | uninterned <i>symbol</i> |
| <b>symbol-namespace</b> <i>sym</i> | <i>ns-designator</i>        |                          |
|                                    | <i>namespace designator</i> |                          |
| <b>symbol-name</b> <i>symbol</i>   | <i>string</i>               | name binding             |
| <b>symbol-value</b> <i>symbol</i>  | <i>T</i>                    | value binding            |

## fixnums

|                                  |               |                            |
|----------------------------------|---------------|----------------------------|
| <b>add</b> <i>fix fix'</i>       | <i>fixnum</i> | sum                        |
| <b>ash</b> <i>fix fix'</i>       | <i>fixnum</i> | arithmetic shift           |
| <b>div</b> <i>fix fix'</i>       | <i>fixnum</i> | quotient                   |
| <b>less-than</b> <i>fix fix'</i> | <i>bool</i>   | <i>fix</i> < <i>fix'</i> ? |
| <b>logand</b> <i>fix fix'</i>    | <i>fixnum</i> | bitwise and                |
| <b>lognot</b> <i>fix</i>         | <i>fixnum</i> | bitwise complement         |
| <b>logor</b> <i>fix fix'</i>     | <i>fixnum</i> | bitwise or                 |
| <b>mul</b> <i>fix fix'</i>       | <i>fixnum</i> | product                    |
| <b>sub</b> <i>fix fix'</i>       | <i>fixnum</i> | difference                 |

## floats

|                                 |              |                          |
|---------------------------------|--------------|--------------------------|
| <b>fadd</b> <i>fl fl'</i>       | <i>float</i> | sum                      |
| <b>fdiv</b> <i>fl fl'</i>       | <i>float</i> | quotient                 |
| <b>fless-than</b> <i>fl fl'</i> | <i>bool</i>  | <i>fl</i> < <i>fl'</i> ? |
| <b>fmul</b> <i>fl fl'</i>       | <i>float</i> | product                  |
| <b>fsub</b> <i>fl fl'</i>       | <i>float</i> | difference               |

## conses/lists

|                               |               |                                      |
|-------------------------------|---------------|--------------------------------------|
| <b>append</b> <i>list</i>     | <i>list</i>   | append lists                         |
| <b>car</b> <i>list</i>        | <i>T</i>      | head of <i>list</i>                  |
| <b>cdr</b> <i>list</i>        | <i>T</i>      | tail of <i>list</i>                  |
| <b>cons</b> <i>T T'</i>       | <i>cons</i>   | ( <i>T</i> . <i>T'</i> )             |
| <b>length</b> <i>list</i>     | <i>fixnum</i> | length of <i>list</i>                |
| <b>nth</b> <i>fix list</i>    | <i>T</i>      | <i>nth</i> <i>car</i> of <i>list</i> |
| <b>nthcdr</b> <i>fix list</i> | <i>T</i>      | <i>nth</i> <i>cdr</i> of <i>list</i> |

## vectors

|                                    |               |                                            |
|------------------------------------|---------------|--------------------------------------------|
| <b>make-vector</b> <i>key list</i> | <i>vector</i> | specialized <i>vector</i> from <i>list</i> |
| <b>vector-length</b> <i>vector</i> | <i>fixnum</i> | length of <i>vector</i>                    |
| <b>vector-type</b> <i>vector</i>   | <i>key</i>    | type of <i>vector</i>                      |
| <b>svref</b> <i>vector fix</i>     | <i>T</i>      | <i>nth</i> element                         |

## namespaces

**mu** (static), **keyword**, **ns**: *#s(:ns #(:t str))*

|                                   |               |                                                     |
|-----------------------------------|---------------|-----------------------------------------------------|
| <b>make-namespace</b> <i>str</i>  | <i>ns</i>     | make <i>namespace</i>                               |
| <b>namespace-name</b> <i>ns</i>   | <i>string</i> | <i>namespace</i> name                               |
| <b>intern</b> <i>ns str value</i> | <i>symbol</i> | intern <i>symbol</i> in non-static <i>namespace</i> |
| <b>find-namespace</b> <i>str</i>  | <i>ns</i>     | map <i>string</i> to <i>namespace</i>               |
| <b>find</b> <i>ns string</i>      | <i>symbol</i> | map <i>string</i> to <i>symbol</i>                  |

## structs

|                                    |               |                                  |
|------------------------------------|---------------|----------------------------------|
| <b>make-struct</b> <i>key list</i> | <i>struct</i> | type <i>key</i> from <i>list</i> |
| <b>struct-type</b> <i>struct</i>   | <i>key</i>    | <i>struct</i> type <i>key</i>    |
| <b>struct-vec</b> <i>struct</i>    | <i>vector</i> | of <i>struct</i> members         |

## streams

|                                            |                        |                                                                          |
|--------------------------------------------|------------------------|--------------------------------------------------------------------------|
| <b>*standard-input*</b>                    | <i>stream</i>          | std input <i>stream</i>                                                  |
| <b>*standard-output*</b>                   | <i>stream</i>          | std out <i>stream</i>                                                    |
| <b>*error-output*</b>                      | <i>stream</i>          | std error <i>stream</i>                                                  |
| <b>open</b> <i>type dir str bool</i>       | <i>stream</i>          | open <i>stream</i> , raise error if <i>bool</i>                          |
|                                            | <i>type</i> <i>dir</i> | <i>:file</i> <i>:string</i> <i>:input</i> <i>:output</i> <i>:bidir</i>   |
| <b>close</b> <i>stream</i>                 | <i>bool</i>            | close <i>stream</i>                                                      |
| <b>openp</b> <i>stream</i>                 | <i>bool</i>            | is <i>stream</i> open?                                                   |
| <b>flush</b> <i>stream</i>                 | <i>bool</i>            | flush <i>stream</i>                                                      |
| <b>get-string</b> <i>stream</i>            | <i>string</i>          | from <i>string</i> <i>stream</i>                                         |
| <b>read-byte</b> <i>stream bool T</i>      | <i>byte</i>            | read <i>byte</i> from <i>stream</i> , error on eof, <i>T</i> : eof-value |
| <b>read-char</b> <i>stream bool T</i>      | <i>char</i>            | read <i>char</i> from <i>stream</i> , error on eof, <i>T</i> : eof-value |
| <b>unread-char</b> <i>char stream char</i> |                        | push <i>char</i> onto <i>stream</i>                                      |
| <b>write-byte</b> <i>byte stream</i>       | <i>byte</i>            | write <i>byte</i>                                                        |
| <b>write-char</b> <i>char stream</i>       | <i>char</i>            | write <i>char</i>                                                        |
| <b>read</b> <i>stream bool T</i>           | <i>T</i>               | read <i>stream</i>                                                       |
| <b>write</b> <i>T bool stream</i>          | <i>T</i>               | write with escape                                                        |

## exceptions

**with-exception** *fn fn'* *T* catch exception

```
fn - (:lambda (obj cond src) . body)
fn' - (:lambda () . body)
```

**raise** *T T'* keyword

raise exception on *T* from  
source designator *T'* with  
keyword condition

```
:arity :div0 :eof :error :except
:future :ns :open :over :quasi
:range :read :exit :signal :stream
:syntax :syscall :type :unbound :under
:write :storage :user
```

## Features

[features]

default = [ "core", "env", "system" ]

|                |                           |             |                   |
|----------------|---------------------------|-------------|-------------------|
| feature/core   | <b>core-info</b>          | list        | core state        |
|                | <b>process-fds</b>        | list        | file descriptors  |
|                | <b>process-mem-virt</b>   | fixnum      | vmem              |
|                | <b>process-mem-res</b>    | fixnum      | reserve           |
|                | <b>process-time</b>       | fixnum      | microseconds      |
| feature/env    | <b>time-units-per-sec</b> | fixnum      |                   |
|                | <b>sleep</b>              | fixnum      |                   |
|                | <b>sleep</b>              | fixnum      |                   |
|                | <b>sleep</b>              | fixnum      |                   |
|                | <b>sleep</b>              | fixnum      |                   |
| feature/system | <b>env-info</b>           | list        | env info          |
|                | <b>heap-info</b>          | list        | list of heap info |
|                | <b>heap-room</b>          | key         | allocations       |
|                | <b>heap-room</b>          | key         | allocations       |
|                | <b>heap-room</b>          | key         | allocations       |
| feature/socket | <b>heap-size</b>          | key         | type size         |
|                | <b>cache-room</b>         | vector      | allocations       |
|                | <b>cache-room</b>         | vector      | allocations       |
|                | <b>cache-room</b>         | vector      | allocations       |
|                | <b>cache-room</b>         | vector      | allocations       |
| feature/system | <b>load</b>               | string      | load mu source    |
|                | <b>symbols</b>            | ns          | list              |
|                | <b>symbols</b>            | ns          | list              |
|                | <b>symbols</b>            | ns          | list              |
|                | <b>symbols</b>            | ns          | list              |
| feature/system | <b>uname</b>              | :t          | system info       |
|                | <b>shell</b>              | string list | shell command     |
|                | <b>exit</b>               | fixnum      | doesn't return    |
|                | <b>sysinfo</b>            | :t          | not on macOS      |
|                | <b>sysinfo</b>            | :t          | not on macOS      |
| feature/socket | <b>accept</b>             |             |                   |
|                | <b>bind</b>               |             |                   |
|                | <b>connect...</b>         |             |                   |
|                | <b>connect...</b>         |             |                   |
|                | <b>connect...</b>         |             |                   |

## environment config

JSON config format:

```
{
  "pages": N,
  "gc-mode": "none" | "auto",
}
```

## Mu library API

[dependencies]

```
mu = {
  git = "https://github.com/Software-Knife-and-Tool/mu.git",
  branch = "main"
}

use mu::{ Mu, Env, Config };
use mu::{ Result, Tag, Exception, Condition };

impl Mu {
  fn compile(_: &Env, _: Tag) → Result<Tag>
  fn config(_: Option<String>) → Config
  fn env(_: &Config) → Env
  fn eq(_: Tag, _: Tag) → bool;
  fn err_out() → Tag
  fn eval_str(_: &Env, _: &str) → Result<Tag>
  fn eval(_: &Env, _: Tag) → Result<Tag>
  fn exception_string(_: &Env, _: Exception) → String
  fn load(_: &Env, _: &str) → Result<bool>
  fn env(_: &Config) → Env
  fn nil() → Tag
  fn read_str(_: &Env, _: &str) → Result<Tag>
  fn read(_: &Env, _: Tag, _: bool, _: Tag) → Result<Tag>
  fn std_in() → Tag
  fn std_out() → Tag
  fn version() → &str
  fn write_str(_: &Env, _: &str, _: Tag) → Result<()>
  fn write_to_string(_: &Env, _: Tag, _: bool) → String
  fn write(_: &Env, _: Tag, _: bool, _: Tag) → Result<()>
}
```

API reference:

```
compile &Env T → Result<Tag>
config Option<String> → Config
env &Config → Env
eq T T' → bool
err_out → Tag
eval_str &Env &str → Result<Tag>
eval &Env T → Result<Tag>
exception_string &Env Exception → String
load &Env &str → Result<bool>
env &Config → Env
nil → Tag
read_str &Env &str → Result<Tag>
read &Env stream bool eof-value → Result<Tag>
// bool – raise exception on end of stream
std_in → Tag
std_out → Tag
version → &str
write_str &Env &str stream → Result<()>
write_to_string &Env T bool → String
// bool – print escaped
write &Env T bool stream → Result<()>
// bool – print escape
```

## Reader Syntax

```
; comment to end of line
#|...|# block comment
'form quoted form
`form backquoted form
`(...) backquoted list (proper lists)
,form eval backquoted form
,@form eval-splice backquoted form
(...) constant list
() empty list, prints as :nil
(...) .) dotted list
"..." string, char vector
| single escape in strings
ns:name qualified symbol, where ns and name are symbol constituents
name lexical symbol
#* bit vector
#x hexadecimal fixnum
#. read-time eval
#\ char
#(:type ...) vector
#s(:type ...) struct
#:... uninterned symbol
"; terminating macro char
# non-terminating macro char

!$%&*+-. symbol constituent
<=>=?@[|
:^_{}~/
A..Za..z0..9

0x09 #\tab character designators
0x0a #\linefeed
0x0c #\page
0x0d #\return
0x20 #\space
```